INFORME TÉCNICO DETALLADO

Construcción de un Observatorio de Educación Superior (OES): El marketing en la educación

Práctica Guiada N.° 08 · Proceso, decisiones y diseño técnico

Curso: Inteligencia Artificial aplicada al Marketing Digital

Universidad Nacional de Cañete (UNDC)

Integrantes: [Completar nombres de los integrantes]

Fecha: 13/11/2026

App pública: https://ad0v1-oes-observatorio-practica08-app-qcrrse.streamlit.app/

Repositorio: https://github.com/Ad0v1/oes-observatorio-practica08

Índice de secciones

- 1. Portada e índice
- 2. Resumen ejecutivo
- 3. Análisis de la guía y sus 12 observaciones
- 4. Planificación y organización del trabajo
- 5. Entorno de desarrollo
- 6. Diseño técnico
- 7. Obtención de datos reales
- 8. Pruebas
- 9. Auditoría
- 10. Publicación
- 11. Limitaciones, riesgos y trabajo futuro
- 12. Conclusiones y lecciones aprendidas
- 13. Anexos

Este documento explica con detalle qué se hizo y por qué. Es distinto del informe de cumplimiento del laboratorio (Informe_OES_Practica08.docx), que responde a la rúbrica de la guía. Todo dato aquí proviene de los archivos del proyecto: la guía, el contrato técnico, las entregas de cada agente, el código y las evidencias.

2. Resumen ejecutivo

Se construyó un prototipo académico de Observatorio de Educación Superior (OES) con Python 3.11, SQLite y Streamlit, siguiendo la Práctica Guiada N.° 08. El sistema registra noticias de instituciones de educación superior peruana con fuente, fecha, URL directa, tema de comunicación y estado de verificación; permite filtrarlas, calcular métricas, graficarlas, exportarlas a CSV y leer una matriz institución × tema como enfoque de marketing.

El trabajo lo realizó un equipo de agentes de IA coordinado por un orquestador, sin subagentes y con participaciones de máximo 8 minutos, comunicados por archivos (un contrato técnico y entregas). Se desarrolló en cuatro rondas. Se corrigieron 12 problemas de la guía, entre ellos marcadores de URL que podían insertarse por error, la ruta relativa de la base y el uso obsoleto de use_container_width.

La base final tiene 7 noticias: 3 DEMO de la guía («Ejemplo didáctico») y 4 reales en estado «Verificada» (1 de UNDC y 3 de PUCP). Las métricas del tablero son 7 noticias, 4 instituciones, 3 categorías y 4 verificadas. La suite automática pasó 13 de 13 pruebas y las seis pruebas manuales P1–P6 de la guía resultaron satisfactorias. Un auditor estimó 15,5/20 sobre un estado intermedio; los faltantes que señaló (README, informe .docx, tabla de pruebas, altair en requirements, control de registro) se resolvieron después.

La app está publicada en Streamlit Community Cloud en modo solo lectura y el código en un repositorio público de GitHub. Limitaciones principales: gob.pe bloqueó el acceso automatizado (HTTP 418) y no se eludió, por lo que no hay noticias reales de entes rectores o reguladores; la verificación la hicieron agentes de IA y requiere confirmación humana del grupo; la nube no conserva altas hechas desde la app.

3. Análisis de la guía y sus 12 observaciones

3.1 Qué pedía la guía

La guía (Práctica Guiada N.° 08, «El marketing en la educación») pedía construir un prototipo local de observatorio con estos elementos:

- Objetivo: diseñar, implementar y validar un prototipo que registre, clasifique, consulte y visualice novedades de educación superior peruana con trazabilidad de fuentes.
- Requisitos RF01–RF04 (registrar, filtrar por categoría y texto, calcular indicadores, exportar CSV) y RNF01–RNF02 (trazabilidad y ética; funcionamiento local sin servicios de pago).
- Entorno: Python, entorno virtual, pip install streamlit pandas.
- Diseño de datos: entidades Institución, Fuente y Noticia, pero con una tabla de noticias desnormalizada para simplificar.
- Código: preparar_datos.py (3 registros DEMO), app.py (dashboard) y un Paso 7 para sustituir ejemplos por registros verificados con URL directa.
- Pruebas P1–P6: carga doble, filtro Universidad pública, búsqueda PUCP, término inexistente, exportar CSV, agregar noticia real.

- Niveles de ampliación: básico (obligatorio), intermedio (normalizar, fecha de captura, verificado_por, estado, informe PDF, filtro por rango de fechas) y avanzado (transparencia presupuestal, alertas, IA con revisión humana).
- Rúbrica de 20 puntos: arquitectura 4, base de datos y trazabilidad 4, aplicación 5, pruebas 4, análisis crítico 3.

3.2 Las 12 observaciones y correcciones

Al analizar la guía se detectaron 12 puntos a corregir o completar. La tabla indica el problema, la corrección y dónde se aplicó.

N.°	Problema detectado	Corrección	Dónde se aplicó
1	El subtítulo «El marketing en la educación» no tenía desarrollo.	Campo tema y matriz institución × tema.	oes_db.TEMAS; columna tema; pestaña Panorama de app.py
2	La figura de la guía mostraba Instituciones/Fuentes/Noticias, pero se implementa una sola tabla.	Nuevo diagrama que distingue lo implementado de lo propuesto.	informe/arquitectura.png (generar_diagrama.py)
3	«Categoría» contiene el tipo de institución, no un tema.	Se mantiene el nombre por compatibilidad con P2 y se añade tema como clasificación temática.	Esquema y constantes de oes_db.py
4	RF01 «Registrar noticia» no tenía interfaz.	Formulario Registrar y script agregar_noticia.py.	app.py (pestaña Registrar); agregar_noticia.py
5	El Paso 7 usaba la ruta relativa oes.db: ejecutado desde otra carpeta crea una base vacía.	Ruta absoluta junto al módulo y variable de entorno OES_DB.	oes_db.ruta_db()
6	El marcador https://URL-DIRECTA-DE-LA-NOTA podía insertarse por error e INSERT OR IGNORE ocultaba duplicados.	Validación que rechaza marcadores e informa duplicados.	oes_db.validar_noticia / insertar_noticia
7	use_container_width está obsoleto en Streamlit reciente.	Se usa width="stretch".	app.py
8	La fecha como texto libre rompe el orden.	CHECK en SQLite y validación AAAA-MM-DD no futura.	ESQUEMA y validar_noticia
9	En PowerShell la activación del entorno puede bloquearse; el comando python del equipo era 3.8.	Nota Set-ExecutionPolicy -Scope CurrentUser RemoteSigned; se usó py -3.11.	README.md; entorno (sección 5)
10	Versiones sin fijar.	requirements.txt con versiones fijas.	requirements.txt
11	Trazabilidad básica, sin fecha de captura ni verificador.	Campos fecha_captura, verificado_por y estado_verificacion.	Esquema; filtro por estado en app.py
12	P6 no era reproducible entre grupos.	Se guarda la URL directa y la captura de la nota original.	datos/*.csv; evidencias oes-07 y oes-08

Cabe señalar que algunas mejoras de nivel intermedio de la guía (fecha de captura, verificado_por, estado, filtro por fechas) se adoptaron en el diseño base y no como extensión posterior.

4. Planificación y organización del trabajo

4.1 Equipo de agentes de IA

El equipo se organizó en roles. La columna de tiempo muestra los minutos que cada agente declaró en su entrega; «n/d» indica que la entrega no los registra.

Agente	Rol	Entradas	Salidas	Tiempo
A3	Datos	Contrato (esquema, API)	oes_db.py, preparar_datos.py, agregar_noticia.py, oes.db	~2 min
A4	Dashboard	Contrato, sección 5	app.py (r1); gráficos Altair y matriz solo verificadas (r2)	~3 min (r1); ~3 min (r2)
A5	Pruebas	Contrato, código de A3 y A4	tests/test_oes.py (13 pruebas)	~2 min
A6a	Investigador: entes	Contrato, sección 6	datos/noticias_entes.csv (solo cabecera)	~3 min
A6b	Investigador: universidades	Contrato, sección 6	datos/noticias_univ.csv (4 noticias)	~8 min
A8	Diagrama	Contrato, guía	informe/arquitectura.png y generar_diagrama.py	~3 min
A9	Informe	Contrato, guía, resultados	contenido_informe.md (r1); Informe_OES_Practica08.docx (r2)	~4 min (r1); r2 n/d
A10	Auditor	contenido_informe.md, código, base	Puntaje estimado 15,5/20 y hallazgos H1–H13	~4 min
Orquestador	Coordinación y verificación	Entregas, navegador, guía	contrato.md, requirements.txt, decisiones, capturas, publicación	n/d

4.2 Restricciones

- Sin subagentes: ningún agente podía crear otros agentes ni delegar su tarea.
- Máximo 8 minutos por participación: se consultaba la hora con Get-Date al empezar y al minuto 6 el agente dejaba de avanzar y escribía su entrega.
- Cada agente toca solo sus archivos; las inconsistencias del contrato se anotan como bloqueo, no se modifican.
- Python del proyecto: siempre el del entorno virtual (3.11), nunca «python» a secas (3.8).

4.3 Comunicación por pizarra

Los agentes no se hablaban entre sí. Se comunicaban mediante archivos compartidos, a modo de pizarra:

- coordinacion/contrato.md: fuente única de verdad. Define estructura de archivos, esquema SQLite, valores permitidos, API de oes_db.py, comportamiento obligatorio de app.py (claves de widgets como f_categoria, orden de métricas), formato del CSV y reglas del equipo. Solo el orquestador lo modifica.
- coordinacion/entregas/<ID>_r<N>.md: cada agente informa Hecho, Archivos, Cómo verificar, Pendiente, Bloqueos y Minutos usados.

Este esquema permitió trabajar en paralelo contra una interfaz común: A4 y A5 usaron la API de A3 tal como estaba en el contrato. Un caso lo ilustra: A5 detectó que en app.py la primera tabla (dataframe[0])

es la matriz institución × tema y no la de noticias, y lo comunicó en su entrega; la prueba se ajustó para buscar la tabla con columna url.

4.4 Las cuatro rondas

Ronda	Qué corrió en paralelo	Por qué
R1	Datos, app, diagrama, investigación de noticias, borrador del informe y pruebas.	Eran tareas independientes porque el contrato fijaba esquema, API y comportamiento; así se aprovechó el límite de 8 minutos.
R2	Pruebas y evidencias.	Dependían de que el código y los datos existieran; se ejecutaron P1–P6 y se capturaron evidencias.
R3	Corrección de gráficos (A4 r2), informe .docx (A9 r2) y auditoría (A10).	Los defectos visuales aparecieron al verificar en navegador; informe y auditoría podían avanzar a la vez sobre un estado ya estable.
R4	Correcciones finales.	Resolver los hallazgos del auditor: README, altair en requirements, control de registro por clave.

4.5 Papel del orquestador

El orquestador no escribió el grueso del producto, pero tomó las decisiones que los agentes no podían tomar por sí mismos:

- Redactar y mantener el contrato técnico (único punto de verdad) y crear requirements.txt.
- Revisar cada entrega y resolver bloqueos (por ejemplo, el contrato no incluía el texto de la guía, y A9 tuvo que inferir RF/RNF y P1–P6 en su primera ronda; en la segunda se usó el texto exacto).
- Verificar en navegador lo que las pruebas automáticas no ven: allí se descubrió que los gráficos hacían zoom con la rueda del ratón.
- Confirmar en el DOM de la página de la UNDC la fecha de la nota CEPRE, que el agente A6b había dejado por confirmar.
- Decidir qué datos aceptar o descartar, ejecutar las pruebas de la guía, publicar en GitHub y Streamlit Cloud, y declarar las limitaciones.

5. Entorno de desarrollo

El comando python del equipo apunta a Python 3.8, que no es compatible con las versiones fijadas de Streamlit y pandas. Se usó Python 3.11 mediante el lanzador de Windows (py -3.11) para crear un entorno virtual .venv, y los agentes ejecutaron el código con .venv\Scripts\python.exe. La nota sobre la política de ejecución de PowerShell se recogió en el README.

```
py -3.11 -m venv .venv
.venv\Scripts\activate
python -m pip install -r requirements.txt
```

Versiones fijas (requirements.txt):

Paquete	Versión	Nota
streamlit	1.67.0	Interfaz y AppTest para pruebas
pandas	3.1.1	Lectura SQL, filtros, tablas cruzadas
altair	6.3.0	Se añadió tras la auditoría: app.py lo importa y no estaba declarado

Observación: el entorno del proyecto no tenía matplotlib, por lo que el diagrama (A8) se generó con py -3.11.

6. Diseño técnico

6.1 Arquitectura

El flujo general es fuente → captura → validación → almacenamiento → consulta → decisiones. La captura es manual (formulario, script o CSV), la validación y el almacenamiento están centralizados en oes_db.py y la consulta ocurre en el dashboard de app.py. El diagrama distingue lo implementado de lo propuesto (captura automática, normalización en tres entidades, resúmenes asistidos por IA, análisis prospectivo), que no se implementó.

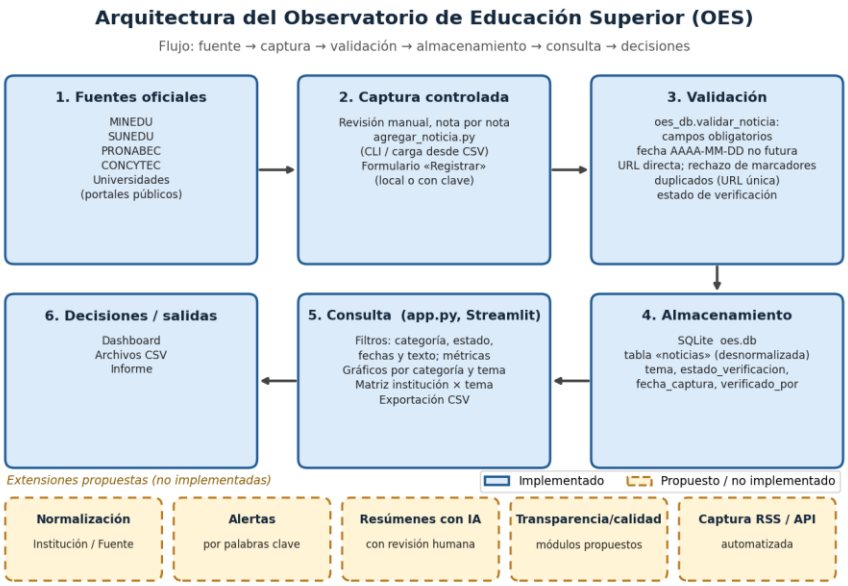


Figura 1. Arquitectura del prototipo: lo implementado (borde continuo) frente a lo propuesto (discontinuo).

Decisión de diseño: oes_db.py es el único módulo que toca SQLite. app.py, preparar_datos.py, agregar_noticia.py y las pruebas lo usan por su API, de modo que las reglas de validación existen en un solo lugar y se aplican igual desde la web, la línea de comandos y la carga CSV.

6.2 Modelo de datos

Una sola tabla desnormalizada «noticias», como propone la guía por simplicidad. El costo es repetir institución, categoría y fuente en cada fila; la normalización en Institución, Fuente y Noticia queda como trabajo futuro.

Columna	Tipo	Restricción	Propósito
id	INTEGER	PRIMARY KEY AUTOINCREMENT	Identificador
titulo	TEXT	NOT NULL	Titular de la nota
institucion	TEXT	NOT NULL	Institución que publica
categoria	TEXT	NOT NULL	Tipo de institución (nombre conservado por compatibilidad con P2)
fuente	TEXT	NOT NULL	Medio o portal de origen
fecha	TEXT	NOT NULL, CHECK (fecha = date(fecha))	Fecha de publicación AAAA-MM-DD
url	TEXT	NOT NULL UNIQUE	URL directa; impide duplicados
resumen	TEXT	NOT NULL	Resumen de redacción propia
tema	TEXT	NOT NULL DEFAULT 'Sin tema'	Tema de comunicación (marketing)
estado_verificacion	TEXT	NOT NULL DEFAULT 'Ejemplo didáctico', CHECK IN (Ejemplo didáctico, Verificada, Pendiente)	Trazabilidad
fecha_captura	TEXT	NOT NULL DEFAULT hoy	Cuándo se registró
verificado_por	TEXT	(nulo permitido)	Quién verificó

Si la base existe con una versión anterior, crear_esquema agrega las columnas nuevas con ALTER TABLE. SQLite no admite un DEFAULT no constante en ADD COLUMN, por lo que en la migración fecha_captura usa un literal; las inserciones siempre fijan la fecha real.

Valores permitidos: categorías «Ente rector», «Ente regulador», «Universidad pública» y «Universidad privada»; ocho temas (Admisión y captación, Becas y financiamiento, Calidad y licenciamiento, Investigación e innovación, Imagen institucional y eventos, Normativa y políticas, Alianzas y convenios, Sin tema); y tres estados.

6.3 API de oes_db.py

Función	Qué hace
ruta_db()	Devuelve la ruta de la base: variable OES_DB si existe; si no, oes.db junto al módulo. Se evalúa en cada llamada (las pruebas cambian la variable).
conectar(ruta)	Abre una conexión SQLite.
crear_esquema(con)	Crea la tabla y agrega columnas nuevas si la base es antigua.
insertar_demo(con)	Inserta los 3 registros DEMO con INSERT OR IGNORE (idempotente).
validar_noticia(d)	Devuelve la lista de errores en español; vacía si es válida.
insertar_noticia(d, ruta)	Valida e inserta; devuelve (éxito, mensaje). Informa «Duplicada» si la URL ya existe.
leer_noticias(ruta)	DataFrame ordenado por fecha e id descendentes; vacío con columnas si no hay datos.
filtrar(df, ...)	Aplica categorías, estados, texto y rango de fechas; None = sin filtro, lista vacía = 0 filas.
a_csv(df)	Serializa a CSV UTF-8 con BOM (se abre bien en Excel).

6.4 Reglas de validación

- Obligatorios no vacíos: titulo, institucion, categoria, fuente, fecha, url, resumen.
- Fecha con formato AAAA-MM-DD válido y no futura.
- URL con esquema http o https y dominio; se rechazan los marcadores (URL-DIRECTA, EJEMPLO.COM).
- Se rechazan los textos marcador de la guía (TÍTULO VERIFICADO, INSTITUCIÓN, FUENTE OFICIAL, RESUMEN REDACTADO Y VERIFICADO), comparados sin mayúsculas ni espacios.
- Categoría, tema y estado deben pertenecer a las listas permitidas (por defecto: tema «Sin tema» y estado «Pendiente»).
- Una noticia «Verificada» exige verificado_por y una URL directa (ruta distinta de vacío o «/»).

```
# oes_db.py (fragmento)
if estado == "Verificada":
    if not v["verificado_por"]:
        err.append("Una noticia Verificada requiere 'verificado_por'")
    if url and urlparse(url).path in ("", "/"):
        err.append("Una noticia Verificada requiere URL directa (no la portada)")
```

6.5 Lógica de la aplicación

- Filtros en la barra lateral, con claves fijas definidas en el contrato para que las pruebas los manejen: categoría (f_categoria), estado de verificación (f_estado), rango de fechas (f_fechas, tolerando una sola fecha) y búsqueda por título o institución (f_busqueda).
- Métricas en orden: Noticias, Instituciones, Categorías, Verificadas. Sin resultados, las métricas muestran 0 y un aviso «No hay resultados con esos filtros», sin excepción.
- Pestañas: Panorama (barras por categoría, barras por tema y matriz institución × tema), Noticias (tabla con enlace «Abrir nota» y botón Exportar CSV), Registrar y Metodología.
- Matriz de marketing solo con noticias verificadas: si el filtro contiene alguna «Verificada», la matriz se calcula únicamente con ellas; si no hay ninguna, usa todas y lo indica en un texto. Así los ejemplos didácticos no contaminan la lectura. Una nota aclara que la matriz describe qué comunica cada institución y no mide calidad.
- La app nunca crea la base: si no existe o está vacía, muestra «No hay noticias. Ejecute preparar_datos.py» y se detiene.

Control del registro. La pestaña Registrar está protegida. La primera versión habilitaba el formulario sin clave si la cabecera Host empezaba por localhost o 127.0.0.1. El auditor señaló que esa cabecera la controla el cliente. La lógica final es:

```
# app.py (fragmento): _registro_habilitado
try: clave = st.secrets["clave_registro"]
except Exception: clave = None
if clave is None:            # sin clave definida: solo local (cabecera host)
    ...
ingresada = st.text_input("Clave de registro", type="password")
if ingresada and hmac.compare_digest(ingresada, str(clave)): ...
```

Es decir: si existe clave_registro en los secrets, siempre se exige; si no existe, el formulario solo se habilita cuando el host es local y, en caso contrario, se muestra «Modo solo lectura». En la nube pública no hay secrets definidos, por lo que el registro queda deshabilitado.

6.6 Seguridad

- SQL parametrizado: todos los INSERT y SELECT usan marcadores ?; el único f-string (ALTER TABLE en crear_esquema) usa nombres de columna fijos del diccionario interno, no datos del usuario (comprobado por el auditor).
- Comparación de clave con hmac.compare_digest, que evita ataques de tiempo.
- Sin secretos en el repositorio: la clave se lee de st.secrets y el despliegue público no define ninguno.
- Correo de commits noreply de GitHub, para no exponer correos personales.
- Búsqueda con regex=False: el texto del usuario se trata como literal.

7. Obtención de datos reales

7.1 Método

La guía exige que cada noticia real se verifique contra su URL directa: titular y fecha deben coincidir con la nota. Se aplicó nota por nota: el agente abría la URL directa, leía el titular y la fecha en la propia página, redactaba un resumen propio de una o dos oraciones y dejaba en el CSV una columna evidencia con cómo se comprobó (no se carga a la base). Se limitó el periodo a 2026-09-01 – 2026-11-13. Un resumen de buscador no cuenta como verificación.

7.2 Bloqueo de gob.pe

Las portadas de gob.pe de MINEDU, SUNEDU, PRONABEC y CONCYTEC, y una nota directa de SUNEDU, respondieron HTTP 418 al acceso automatizado. No se intentó eludir el bloqueo. Por eso datos/noticias_entes.csv contiene solo la cabecera y no hay noticias reales de entes. A6a dejó pistas sin verificar (notas de SUNEDU que habrían salido los días 03, 05 y 09 de noviembre de 2026, tomadas de un resumen de buscador); no se registraron.

7.3 Noticias aceptadas y descartadas

Institución	Fecha	Tema	URL
UNDC	2026-10-23	Admisión y captación (CEPRE ciclo ordinario 2026-II)	https://web.undc.edu.pe/el-camino-a-la-undc-comienza-hoy-cepre-undc-incio-el-ciclo-ordinario-2026-ii-con-vacantes-de-ingreso-directo/
PUCP	2026-10-16	Imagen institucional y eventos (Aula Magna 2026)	https://puntoedu.pucp.edu.pe/vida-universitaria/institucional/aula-magna-2026-abrio-el-debate-sobre-crecimiento-economico-desarrollo-humano-e-instituciones/
PUCP	2026-10-30	Investigación e innovación (casa contra heladas con la Universidad Nacional de Juliaca)	https://puntoedu.pucp.edu.pe/investigaciones/impacto-social/pucp-y-universidad-nacional-de-juliaca-unen-esfuerzos-asistir-poblaciones-altoandinas/
PUCP	2026-11-06	Imagen institucional y eventos (honoris causa a Mircea Cărtărescu)	https://puntoedu.pucp.edu.pe/vida-universitaria/institucional/creador-de-un-universo-particular-que-nos-refleja-mircea-cartarescu-doctor-honoris-causa-pucp/

Descartadas por A6b:

- UNMSM, nota de Salamanca: la página no muestra fecha de publicación; sin fecha no cumple la regla de trazabilidad.
- UNDC, nota sobre el Congreso (web.undc.edu.pe): fecha ambigua, por lo que no se aceptó.

Confirmación de la fecha UNDC. La nota CEPRE de la UNDC se aceptó con la fecha tomada del texto visible, sin metadato, y A6b pidió confirmarla. El orquestador la confirmó leyendo el DOM de la página (evidencia oes-08, nota original UNDC).

Salvedades declaradas: el resumen de la nota de Cărtărescu se redactó a partir del titular y los metadatos; los 4 registros fueron verificados por un agente de IA y quedan pendientes de confirmación del grupo; y faltó una segunda universidad pública (el investigador planteaba un mínimo de 2).



Figura 2. Nota original de la UNDC usada para verificar titular y fecha (evidencia oes-08).

8. Pruebas

8.1 Suite automática (pytest)

tests/test_oes.py contiene 13 pruebas y las 13 pasaron. Cada una usa una base temporal (variable OES_DB vía monkeypatch) y ejecuta preparar_datos.py por subproceso; el dashboard se prueba con AppTest de Streamlit.

Prueba	Qué comprueba
test_p1_idempotencia	Dos ejecuciones de preparar_datos.py dejan 3 filas con URLs únicas.
test_p0_metricas_sin_filtros	Sin filtros: 3 noticias, 3 instituciones, 3 categorías y sin excepción.
test_p2_filtro_categoria	Filtrar «Universidad pública» muestra solo UNMSM.
test_p3_busqueda	Buscar PUCP devuelve 1 fila.
test_p4_sin_resultados	Un término inexistente da 0 noticias y un aviso, sin error.
test_p5_csv	El CSV tiene encabezados y solo las filas filtradas (PUCP); si AppTest lo expone, verifica el botón de descarga.
test_p6_insertar_verificada	Insertar una noticia verificada sube a 4 filas, conserva la URL y la métrica Verificadas pasa a 1.
test_rechaza_url_marcador	Se rechaza la URL marcador https://URL-DIRECTA-DE-LA-NOTA.
test_rechaza_fechas	Se rechazan una fecha con formato 12/11/2026 y una fecha futura.
test_rechaza_categoria	Se rechaza una categoría fuera de la lista.
test_verificada_sin_verificador	Una Verificada sin verificado_por no se inserta.
test_duplicada	La segunda inserción de la misma URL se rechaza con «Duplicada».
test_filtro_fechas	El rango 2026-11-06 a 2026-11-06 devuelve 2 filas.

```
python -m pytest tests -q   # 13 passed
```

8.2 Pruebas manuales P1–P6 de la guía

Prueba	Procedimiento	Resultado	Evidencia
P1	Ejecutar la carga dos veces	3 filas, 3 URLs distintas	oes-02-P1-carga-doble.txt
P2	Filtrar Universidad pública	Se muestra UNMSM (dato didáctico)	oes-03-P2-filtro-universidad-publica
P3	Buscar PUCP	1 fila (con los DEMO)	oes-04-P3-busqueda-PUCP
P4	Término inexistente	0 noticias y aviso, sin error	oes-05-P4-termino-inexistente
P5	Exportar CSV	Archivo con encabezados y filas filtradas	oes-06-P5-oes_resultados.csv
P6	Agregar 4 noticias reales	De 3 a 7 filas; URL directa conservada; segunda carga = 4 duplicadas	oes-07-P6-carga-noticias-reales.txt; oes-08; oes-09

La evidencia de P6 muestra que la primera carga registró 4 filas y la segunda las marcó como duplicadas (0 registradas, 4 duplicadas, 0 rechazadas), con el estado final de 7 noticias y 4 verificadas.

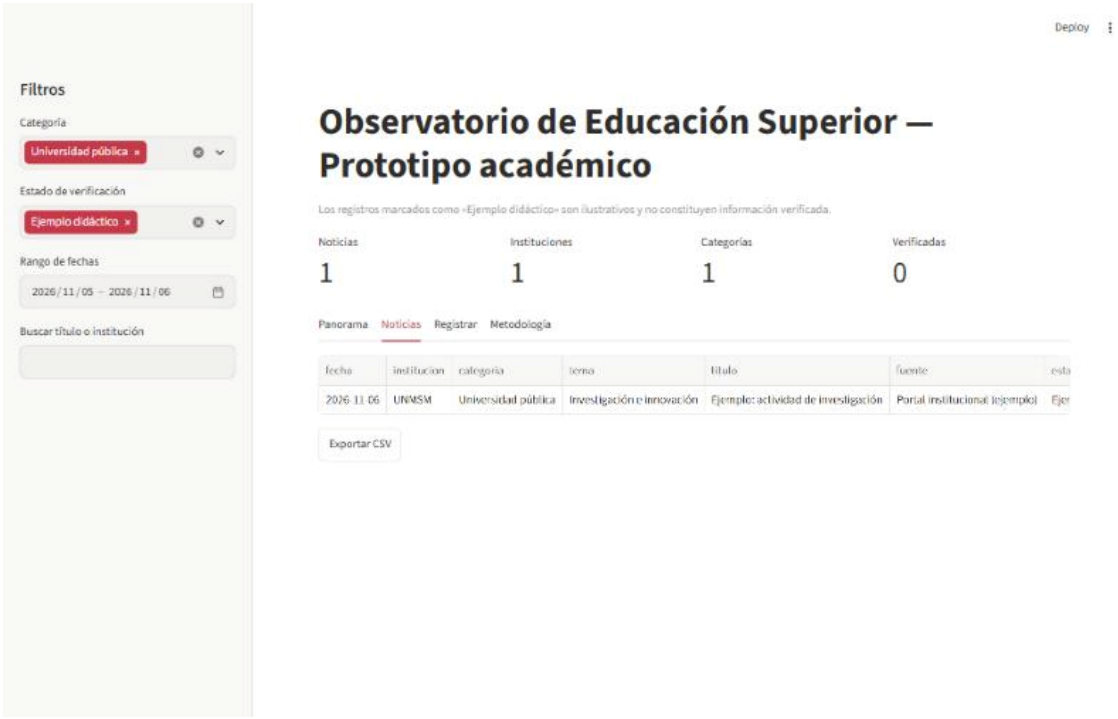


Figura 3. P2: filtro Universidad pública (evidencia oes-03).

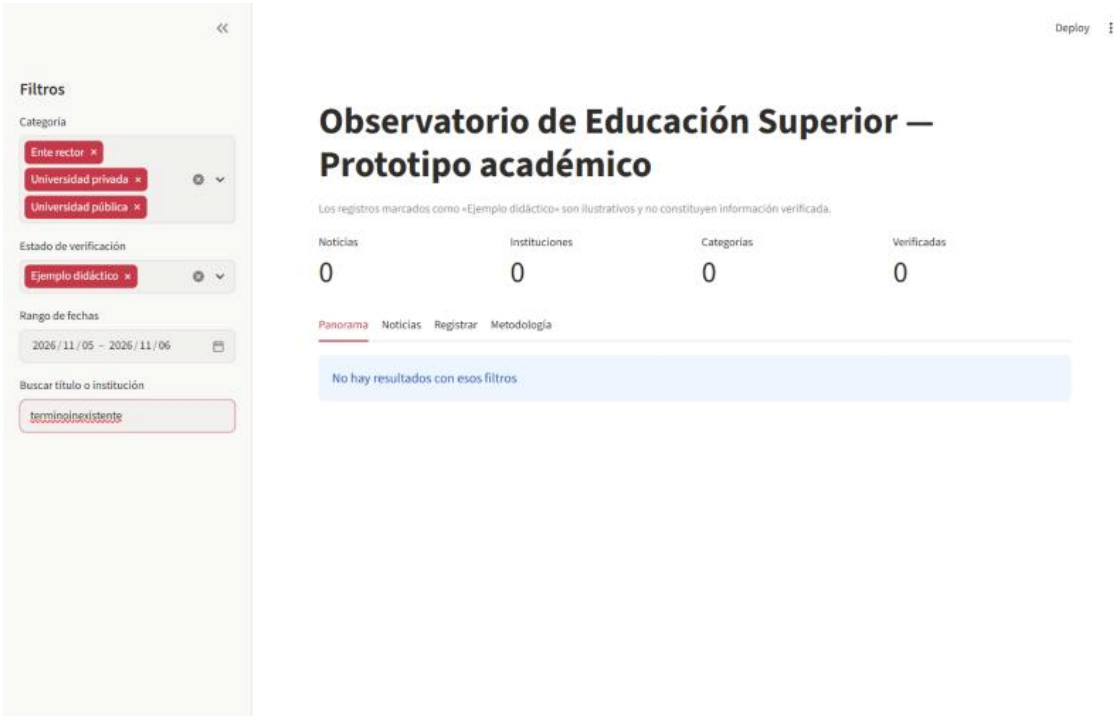


Figura 4. P4: término inexistente con 0 noticias y aviso (evidencia oes-05).

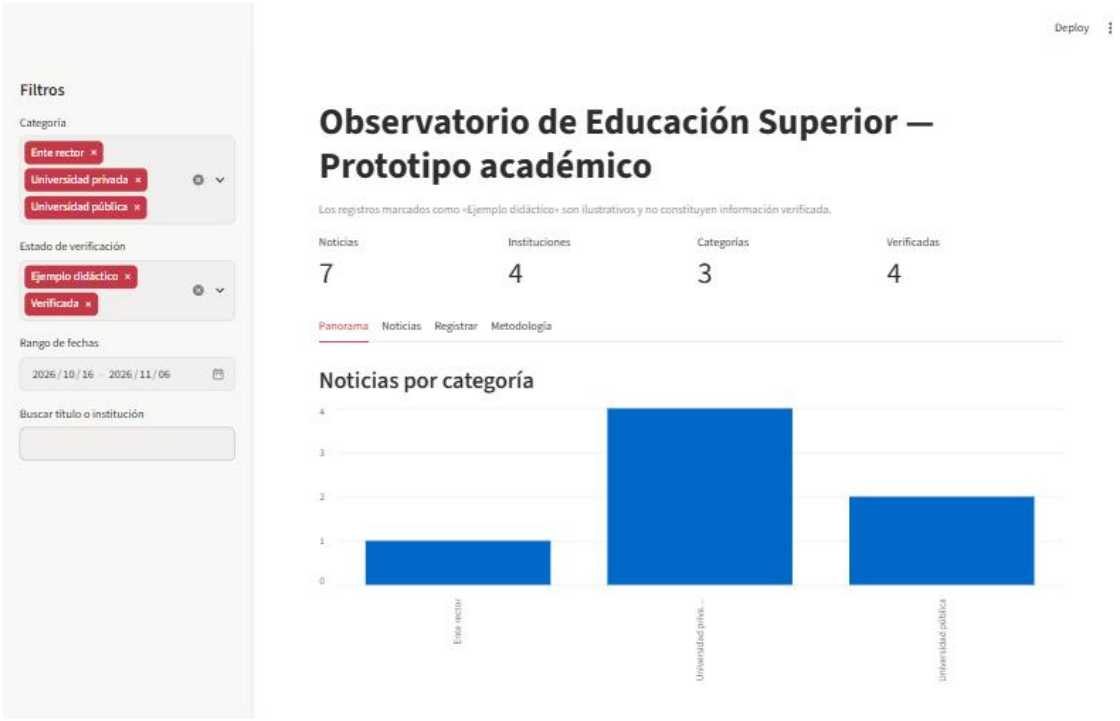


Figura 5. Tablero final con 7 noticias, 4 instituciones, 3 categorías y 4 verificadas (evidencia oes-09).

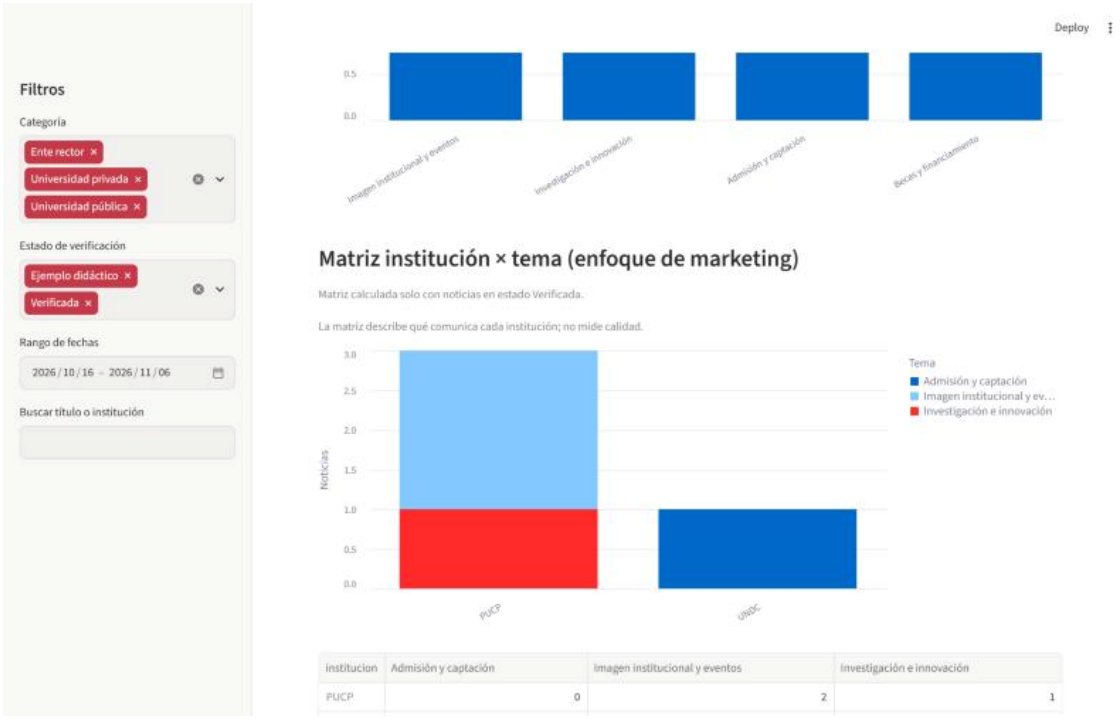


Figura 6. Matriz institución × tema, calculada solo con noticias verificadas (evidencia oes-10).

8.3 Defectos encontrados y corregidos

Defecto	Causa	Solución
Zoom en los gráficos con la rueda del ratón	Los gráficos eran interactivos y la rueda cambiaba la escala al desplazar la página. Se detectó en el navegador, no con pytest.	Gráficos Altair estáticos (sin .interactive()), con etiquetas a -30° y sin truncar, mostrados con st.altair_chart.
altair sin declarar	app.py importa altair, que llegaba solo como dependencia indirecta de Streamlit.	altair==6.3.0 en requirements.txt.
Registro habilitable por cabecera Host	La detección de «local» dependía de una cabecera que controla el cliente.	Si existe clave_registro se exige siempre; sin clave, solo host local; si no, «Modo solo lectura»; comparación con hmac.compare_digest.
Prueba sobre la tabla equivocada	dataframe[0] era la matriz y no la tabla de noticias.	La prueba busca el dataframe con columna url.

9. Auditoría

El agente A10 evaluó el trabajo con la rúbrica de la guía sobre un estado intermedio: revisó contenido_informe.md porque el .docx aún no existía y ejecutó pytest (13 pruebas aprobadas). Estimó 15,5 sobre 20 y concluyó «Listo para entregar: No».

Criterio	Estimado	Máx.
Arquitectura y explicación de flujo	3,5	4
Base de datos y trazabilidad	3,5	4
Aplicación funcional y filtros	4,5	5
Pruebas y evidencias	2,5	4
Análisis crítico, ética y conclusiones	1,5	3
Total	15,5	20

Hallazgos principales y cómo se resolvieron:

Hallazgo	Resolución posterior
H1: las pruebas P1–P6 del borrador no coincidían con las de la guía y estaban en [PENDIENTE].	Se reescribió la tabla con las seis pruebas de la guía y sus evidencias.
H2: el .docx no existía y el borrador tenía marcadores abiertos, sin conclusiones.	Se generó Informe_OES_Practica08.docx con conclusiones y referencias; el único marcador que queda es el de los nombres de los integrantes.
H3: faltaba README.	Se creó README.md con requisitos, comandos, pruebas, estructura y nota de despliegue.
H4: Host controlable por el cliente.	Se invirtió la lógica: clave siempre que exista (sección 6.5).
H5: el disco de Streamlit Cloud es efímero.	Se documentó en README y en el informe; el procedimiento es cargar en local y subir oes.db.
H6: altair sin declarar.	Añadido a requirements.txt.
H7: noticias_entes.csv vacío.	No se pudo resolver por el bloqueo HTTP 418; se declara como limitación.
H8: solo 4 filas verificadas, por IA.	Se declara que «Verificada» significa verificada por un agente de IA, pendiente de confirmación humana.

Entre los hallazgos menores (H9–H13), la captura de P5 es un CSV y no una imagen, y agregar_noticia.py --csv marca todo como «Verificada» sin una opción de estado. Esta última sugerencia no se implementó y queda como mejora.

10. Publicación

10.1 GitHub

El código se subió a un repositorio público (https://github.com/Ad0v1/oes-observatorio-practica08) con el correo noreply de GitHub en los commits. El archivo oes.db se versiona para que la app en la nube lea los 7 registros.

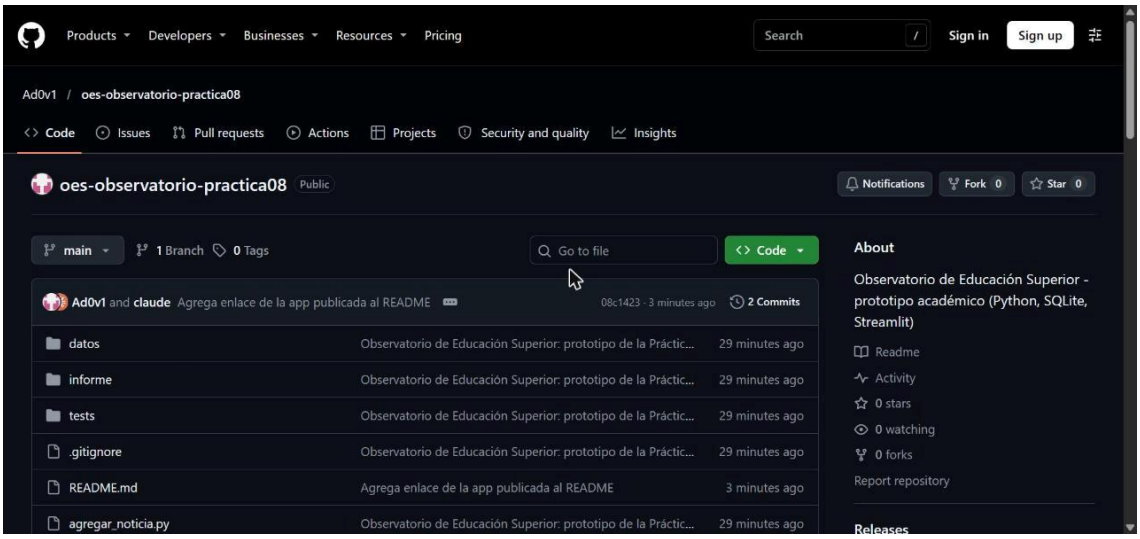


Figura 7. Repositorio público en GitHub (evidencia web-08).

10.2 Streamlit Community Cloud

- Se crea la app desde el repositorio, con app.py como archivo principal.
- Se elige Python 3.11 en la configuración avanzada, igual que en el desarrollo local.
- No se define ningún secret, de modo que el registro queda en «Modo solo lectura».
- Se publica como app pública: https://ad0v1-oes-observatorio-practica08-app-qcrrse.streamlit.app/
- Verificación: en la web, la pestaña Registrar muestra «Modo solo lectura» (evidencia web-05) y el resto de las pestañas funciona (web-01 a web-07).

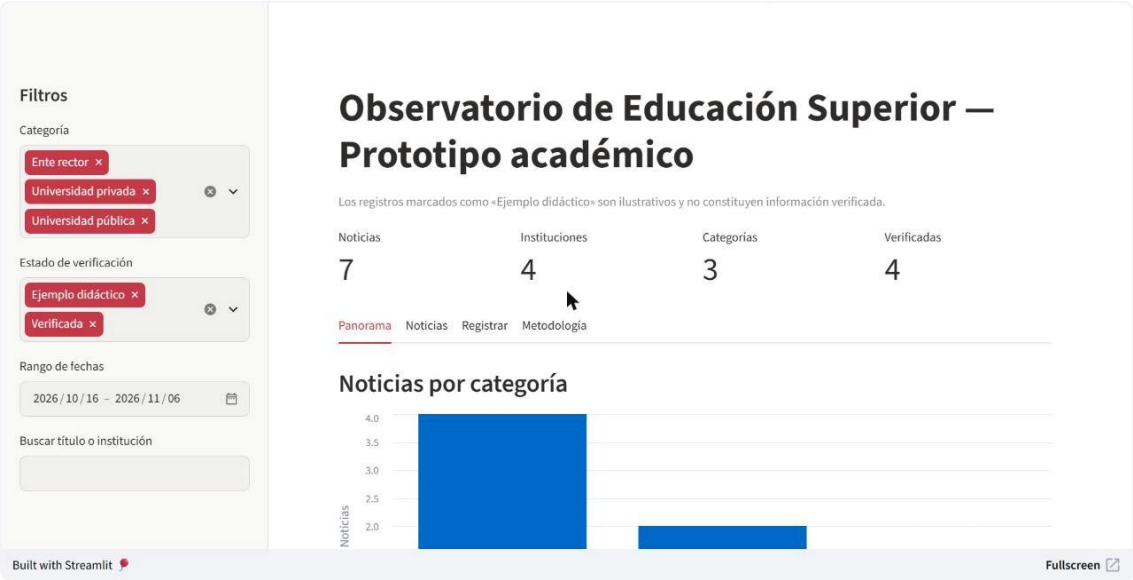


Figura 8. App pública en Streamlit Cloud: pestaña Panorama (evidencia web-01).

Dos comportamientos a tener en cuenta: el disco de la nube no es permanente (lo registrado desde la app se pierde al reiniciar) y la app se «duerme» tras un tiempo sin visitas.

11. Limitaciones, riesgos y trabajo futuro

11.1 Limitaciones

- Muestra pequeña: 4 noticias reales de 2 instituciones (UNDC y PUCP). No se pueden generalizar conclusiones de la matriz.
- Sin noticias reales de entes rectores o reguladores por el bloqueo HTTP 418 de gob.pe, que no se eludió.
- Verificación hecha por agentes de IA; un resumen se basó en titular y metadatos. Requiere confirmación humana del grupo.
- La nube no guarda altas: el flujo es cargar en local y subir oes.db al repositorio.
- Un conteo de noticias no mide calidad ni impacto; la matriz solo describe el mensaje publicado.

11.2 Riesgos

- Dependencia de sitios que cambian su estructura o bloquean accesos automatizados.
- Los registros DEMO apuntan a portadas y podrían confundirse con noticias reales: se marcan «Ejemplo didáctico» y se excluyen de la matriz cuando hay verificadas.
- Datos duplicados por inconsistencia en el nombre de la institución, propia del modelo desnormalizado.

11.3 Trabajo futuro (niveles de la guía)

- Intermedio: normalizar en Institución, Fuente y Noticia y generar informe PDF. Ya están implementados fecha de captura, verificado_por, estado y filtro por rango de fechas.

- Avanzado: módulos de transparencia presupuestal y condiciones de calidad con datos públicos verificables; alertas por palabras clave; resúmenes asistidos por IA con revisión humana; registro de cambios normativos. Indicadores citados en el informe de cumplimiento: PIM/devengado, licenciamiento, acreditación y RENACYT.
- Mejoras puntuales: opción de estado Pendiente en la carga CSV y una base persistente para la nube.

12. Conclusiones y lecciones aprendidas

- El prototipo cumple el objetivo: registra, clasifica, consulta y visualiza novedades con trazabilidad. Las 13 pruebas automáticas y las seis pruebas de la guía resultaron satisfactorias.
- Centralizar la validación en un solo módulo y apoyarla en restricciones de la base (CHECK, UNIQUE) evitó los errores típicos de la guía (marcadores de URL, duplicados ocultos, fechas inválidas).
- Un contrato técnico explícito permitió que agentes con 8 minutos y sin comunicación directa trabajaran en paralelo sin chocar.
- Las pruebas automáticas no sustituyen la revisión visual: el defecto del zoom solo apareció al mirar el navegador.
- La auditoría independiente fue útil: señaló el riesgo de la cabecera Host, el altair sin declarar y la ausencia de README.
- Con agentes de IA es necesario separar «hecho» de «verificado»: los agentes declararon lo que no pudieron comprobar (fechas ambiguas, bloqueo 418, resumen basado en titular), y esas salvedades se trasladaron al informe en lugar de ocultarse.
- Los agentes de IA aceleran la construcción, pero la responsabilidad sobre el contenido final y la confirmación de las fuentes sigue siendo del grupo.

13. Anexos

Anexo A. Estructura de archivos del repositorio

```
oes_laboratorio/
  README.md
  requirements.txt        # streamlit, pandas, altair (versiones fijas)
  oes_db.py               # capa de datos
  preparar_datos.py       # crea oes.db + 3 DEMO
  agregar_noticia.py      # alta por campos o --csv
  app.py                  # dashboard Streamlit
  oes.db                  # base de datos (7 noticias)
  tests/test_oes.py       # 13 pruebas
  datos/                  # noticias_univ.csv, noticias_entes.csv
  informe/                # arquitectura.png, capturas-oes/, capturas-web/, informe
  coordinacion/           # contrato.md y entregas (no entregable)
```

Anexo B. Comandos para reproducir

```
py -3.11 -m venv .venv
.venv\Scripts\activate
```

```
python -m pip install -r requirements.txt
python preparar_datos.py
python agregar_noticia.py --csv datos\noticias_univ.csv
streamlit run app.py                # http://localhost:8501
python -m pytest tests -q           # 13 passed
```

Alta de una noticia por campos:

```
python agregar_noticia.py --titulo "..." --institucion "SUNEDU" --categoria "Ente regulador"
  --fuente "..." --fecha 2026-11-10 --url "https://..." --resumen "..."
  --tema "Calidad y licenciamiento" --verificado-por "Nombre del integrante"
```